CRYPTONIX SYSTEM



Web Security Assessment

TryHackMe — Corridor

MAJOR PROJECT

An Experimental Framework for Object Reference Manipulation and Kernel Exploitation

Prepared by	**Taslim Bano**
Assessment date	**29 October 2026**
Platform	**TryHackMe AttackBox**
Status	**Challenge Completed**

1. Executive Summary

As part of my cybersecurity major project, I investigated the TryHackMe Corridor room to understand how manipulating URL references can reveal resources that are not directly accessible through a website's visible navigation.

The challenge presented a virtual corridor containing multiple doors. Each door was associated with a hexadecimal-looking value in its URL. Rather than treating these values as random identifiers, I examined the application's links and investigated whether the values followed a predictable pattern.

I extracted the available link references from the saved HTML and used Python and the Linux `md5sum` utility to calculate MD5 hashes of numeric strings. By comparing the calculated hashes with the extracted references, I identified a relationship between numeric inputs and the door URLs.

The important discovery came when I calculated the MD5 hash of the string `0` and opened the corresponding endpoint. This revealed the challenge flag:

```
flag{2477ef02448ad9156661ac40a6b8862e}
```

I submitted the flag to TryHackMe and received confirmation that the answer was correct and the room was completed.

This investigation helped me understand an important web security

principle: an identifier that looks random or resembles a hash does not provide access control. If an application generates predictable object references and fails to enforce appropriate authorization, manipulating those references may allow access to unintended resources.

The assessment was limited to the Corridor challenge. Although my broader major project explores web exploitation and privilege escalation, this report documents only the activities and results demonstrated during this investigation.

---

2. Assessment Overview

2.1 Objective

The primary objective was to investigate the URL structure used by Corridor, identify the pattern behind the door references, and retrieve the challenge flag.

The investigation focused on three questions:

- How were the hexadecimal-looking door references generated?
- Could the relationship between the references and numeric inputs be reproduced?
- Would investigating an additional candidate endpoint reveal a

resource that was not represented by the initially visible door links?

2.2 Scope

Item	Details
Target application	TryHackMe Corridor
Target IP observed	10.49.146.177
AttackBox IP observed	10.49.93.230
Operating environment	TryHackMe AttackBox
Browser	Firefox

Analysis tools	Python 3, hashlib, md5sum
Browser inspection	Developer Tools: Network and Storage
Primary technique	Hash analysis and URL reference manipulation
Final outcome	Flag retrieved and room completion confirmed

The IP addresses are the values visible in my screenshots and may change between TryHackMe sessions.

2.3 Methodology

I followed a practical, evidence-driven approach:

1. Inspected the initial corridor interface and its visible doors.
2. Examined the saved HTML and extracted the available link references.
3. Calculated MD5 hashes for numeric strings.

4. Compared the generated hashes against the extracted door references.
5. Investigated the endpoint associated with the numeric input `0`.
6. Inspected the resulting page and relevant HTTP responses.
7. Submitted the flag and verified the room completion status.

I recorded screenshots throughout the investigation so that the main observations and final result could be reviewed independently.

---

3. Technical Findings

F-01: Predictable Hash-Derived Door References

Category: Web application endpoint analysis
Evidence: E03, E04, E05
Result: Pattern identified

Observation

While examining the saved HTML, I found hexadecimal-looking values used as door URL paths.

I extracted the `href` references into a text file named `visible_doors.txt`. The extraction produced 15 link references, including two external stylesheet URLs and 13 hash-like door

references.

I then investigated whether the door references could be reproduced by calculating MD5 hashes of numeric strings.

Technical Analysis

I used `md5sum` to calculate the hashes of the strings `1` and `0`.

Input string	MD5 hash
`1`	`c4ca4238a0b923820dcc509a6f75849b`
`0`	**`cfcd208495d565ef66e7dff9f98764da`**

The hash of `1` matched the first door reference extracted from the HTML.

To investigate the pattern further, I wrote a Python script using the `hashlib` module. The script generated MD5 hashes for candidate numeric inputs and compared them against the extracted reference

set.

The comparison produced the following results:

Candidate indices	Result
1–13	Matching references found
14	No matching reference found

This supported the hypothesis that the door paths were generated by hashing numeric inputs.

Security Implication

The experiment demonstrated that the door identifiers were predictable despite appearing as hexadecimal strings.

In a real web application, predictable identifiers can make endpoint enumeration easier. If the server relies on the difficulty of guessing an identifier instead of checking whether the requesting user is authorized to access the associated resource, this could lead to unauthorized access.

Conclusion

I established the relationship between numeric inputs and the observed door references. This gave me a reproducible way to generate candidate URLs rather than relying exclusively on the links displayed by the application.

F-02: Challenge Flag Retrieved Through the Endpoint Associated with 0

Category: URL reference manipulation
Evidence: E04, E06, E07, E12
Result: Successful

Observation

After identifying the relationship between numeric inputs and the door references, I calculated the MD5 hash of the string `0`.

The resulting hash was:

`cfcd208495d565ef66e7dff9f98764da`

I entered the corresponding path in the browser and observed that the page displayed the challenge flag.

Result

The retrieved flag was:

`flag{2477ef02448ad9156661ac40a6b8862e}`

I submitted this value to the TryHackMe answer field. The platform displayed a correct-answer confirmation and showed 100% room completion.

Security Implication

This result demonstrated that investigating the identifier-generation pattern allowed me to reach the intended challenge resource through a URL that was not among the initially extracted door references.

The important security lesson is that the appearance of an identifier does not establish whether access should be permitted.

A production application must verify authorization for each protected resource request. Hashing an identifier does not replace that check.

Conclusion

I successfully retrieved the flag and completed the Corridor room. The flag-page screenshot and the completion screenshot provide direct evidence of the result.

This demonstrates the intended challenge behavior, although it does not independently establish how authorization is implemented in a

general production application.

F-03: HTTP Response and Browser Storage Inspection

Category: Supporting web application observations
Evidence: E08, E09, E10, E11
Result: Observations recorded

Observation 1: Successful Door Request

I inspected the browser's Network panel while visiting a visible door endpoint:

`/c20ad4d76fe97759aa27a0c99bff6710`

The request returned:

`HTTP 200 OK`

This reference corresponded to candidate index 12 in my Python comparison.

The response confirmed that the tested endpoint returned a successful HTTP response.

Observation 2: Missing Favicon

The Network panel also showed a request for `favicon.ico` returning:

`HTTP 404 Not Found`

This indicated that the requested favicon resource was not found. I did not treat this as a security vulnerability because the response did not demonstrate unauthorized access or exposure of protected information.

Observation 3: Browser Storage

I inspected the browser's Storage panel for the Corridor origin. The selected view displayed no data for the host.

This was recorded as a browser observation. It does not establish that the application never uses client-side storage or that no relevant state exists elsewhere.

Conclusion

These observations provided additional context about the application's behavior during testing. They support the investigation record but do not constitute independent vulnerability findings.

---

4. Evidence Register

The following register maps the screenshots collected during my investigation to the findings they support.

The filenames are standardized evidence names for the repository.

The evidence register links each finding to its supporting artifacts. The screenshots document the extraction process, hash calculations, candidate comparison, endpoint investigation, and final completion result.

---

5. Security Recommendations

Although Corridor is a deliberately designed training challenge, the investigation illustrates several defensive principles relevant to web application development.

Identified concern	Recommended control
Predictable object identifiers	Do not rely on identifier secrecy to protect resources
Potential unauthorized object access	Enforce server-side object-level authorization

Manipulation of URL references	Validate the user's permissions for every protected resource request
Endpoint enumeration	Monitor unusual patterns of repeated or sequential requests
Authorization regressions	Include access-control checks in automated security tests

Example of Server-Side Authorization

The following pseudocode illustrates the principle:

document = load_document(document_id)

if document is None:

return 404

if not user_can_access(current_user, document):

return 403

return document

The critical step is checking whether the current user is authorized to access the requested document.

The identifier format is secondary. Even a randomly generated identifier must not be treated as proof of authorization.

---

6. Limitations

This assessment was limited to the TryHackMe Corridor challenge and the evidence collected during that exercise.

The server-side application source code was not examined, and the complete authorization implementation was not independently verified. Consequently, the findings establish the observed identifier pattern and challenge behavior rather than the security posture of an unrelated production application.

The broader major project explores additional exploitation stages, including web application exploitation, shell execution, Metasploit, and Linux privilege escalation. Those stages were not demonstrated during this Corridor investigation and are not claimed as completed here.

The conclusions in this report are therefore restricted to the techniques performed and the results documented in the evidence

register.

---

7. Conclusion

This investigation helped me understand how a seemingly complex URL can be generated from a simple, predictable input.

By examining the Corridor page, extracting its link references, calculating MD5 hashes, and comparing the results with the observed endpoints, I identified the relationship between numeric strings and the door paths.

I then calculated the hash of `0`, accessed the corresponding endpoint, retrieved the challenge flag, and verified that TryHackMe accepted the answer and marked the room as completed.

The most important lesson was that a hash can obscure an identifier without making it a security boundary. Applications must enforce authorization independently of how resource identifiers are generated.

The Corridor challenge is a focused starting point for my broader cybersecurity project. It provided practical experience in endpoint analysis, hash-based identifier investigation, browser inspection, evidence collection, and documenting security findings.

The challenge was completed successfully, and the evidence

collected provides a reproducible record of the investigation and its outcome.

---

8. References

1. TryHackMe, *Corridor* — challenge environment and instructions.
2. Python Documentation, *`hashlib` — Secure hashes and message digests*.
3. Linux `md5sum` utility documentation.
4. OWASP, *Insecure Direct Object Reference (IDOR) Prevention Cheat Sheet*.